

Object-Oriented Programming in Java

Emily Lucia Antosch

17.09.2026

Today's route

1. Introduction	2
2. Simple Data Types	5
3. Comments and Identifiers	24
4. Operators	32
5. Type Conversion	43
6. Control Structures	51
7. License Notice	74

1. Introduction

Where Are We Now?

Today



The Goal of This Chapter

- Choose Java's primitive data types.
- Control program flow with conditions and loops.
- Write readable code with consistent names and formatting.

2. Simple Data Types

Data Types and Memory



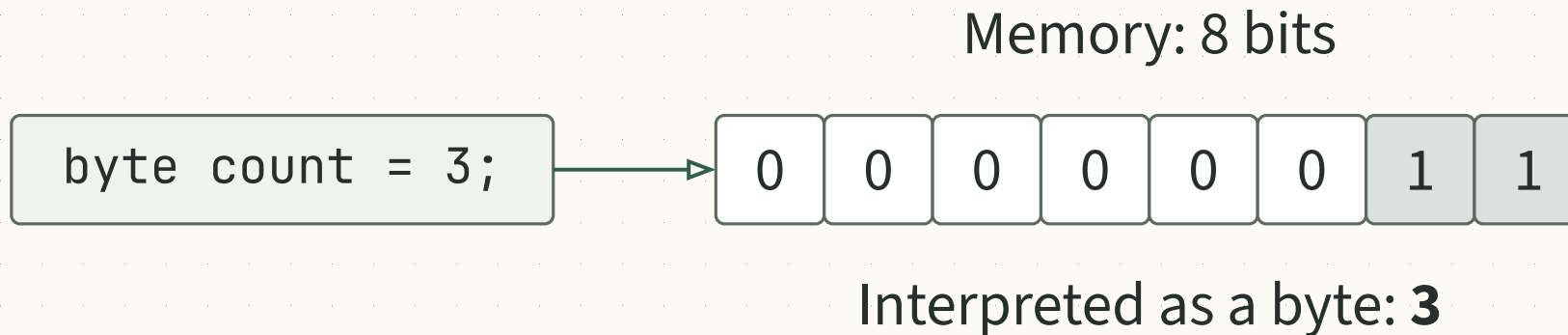
How can a program remember its state?

Data Types and Memory



How can a program remember its state?

Variables store values in memory. Their types determine how those values are interpreted.



Data Types and Memory



Which data types do you know from C?

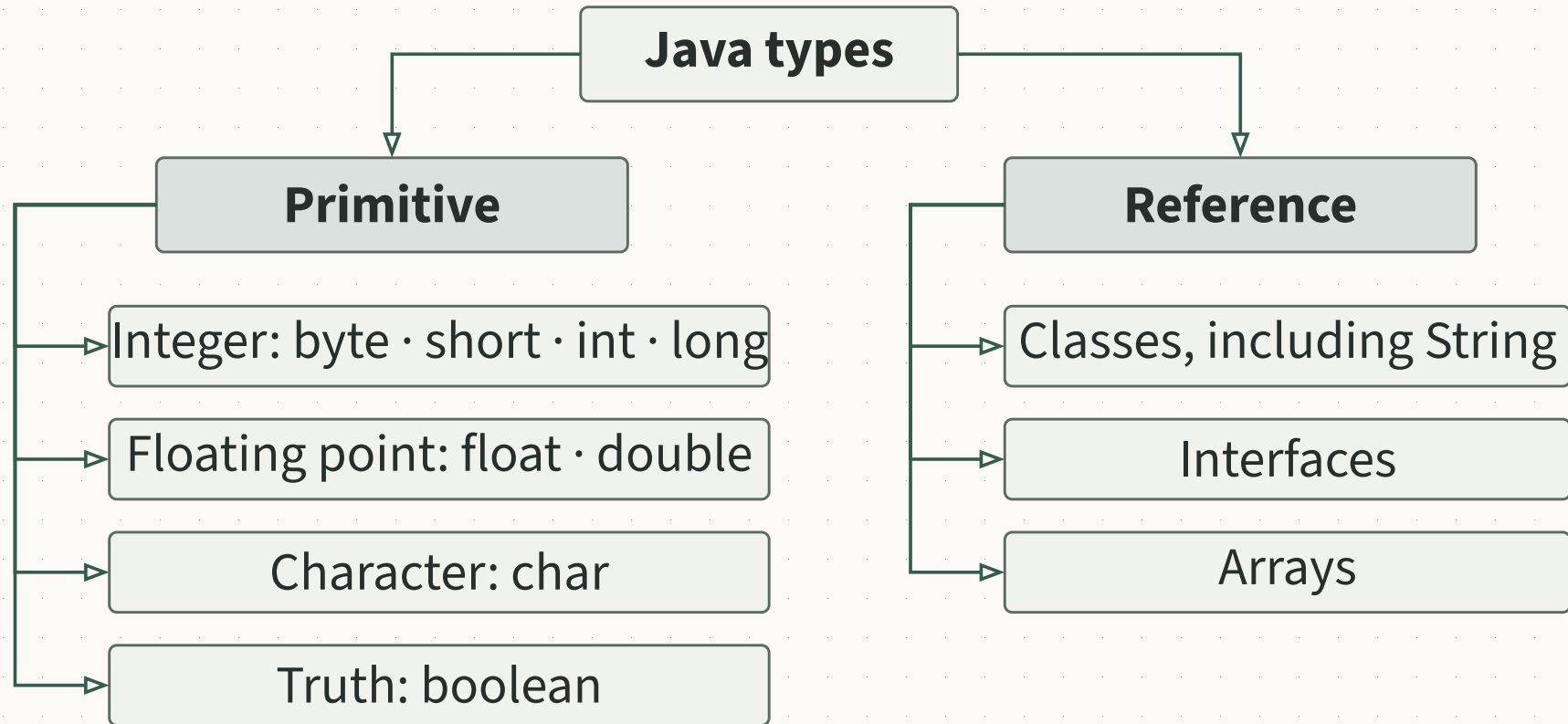
Data Types and Memory



Which data types do you know from C?

- `int`, `char`, `float`, `double`, `bool`
- `struct`, `enum`, `union`
- Arrays and pointers
- `void`: no value

Data Types in Java



Data Types in Java

Kind	Type	Size	Range / values
Integer	byte	1 byte	-2^7 to $2^7 - 1$
	short	2 bytes	-2^{15} to $2^{15} - 1$
	int	4 bytes	-2^{31} to $2^{31} - 1$
	long	8 bytes	-2^{63} to $2^{63} - 1$
Character	char	2 bytes	0 to 65,535 (UTF-16 code unit)
Floating point	float	4 bytes	IEEE 754 binary32
	double	8 bytes	IEEE 754 binary64
Truth	boolean	Not specified	true or false

Declaration and Initialization

Declare

```
int age;
```

Initialize

```
age = 24;
```

Use

```
println(age)
```

One step:

```
int age = 24;
```



Memorize

Declare a local variable before use. Assign a value before reading it.

Declaration and Initialization

Declare first, then assign the initial values with =.

Declaration

```
1 int a;  
2 float b;  
3 char c;
```

Java

Initialization

```
1 a = 5;  
2 b = 3.5f;  
3 c = 'A';
```

Java

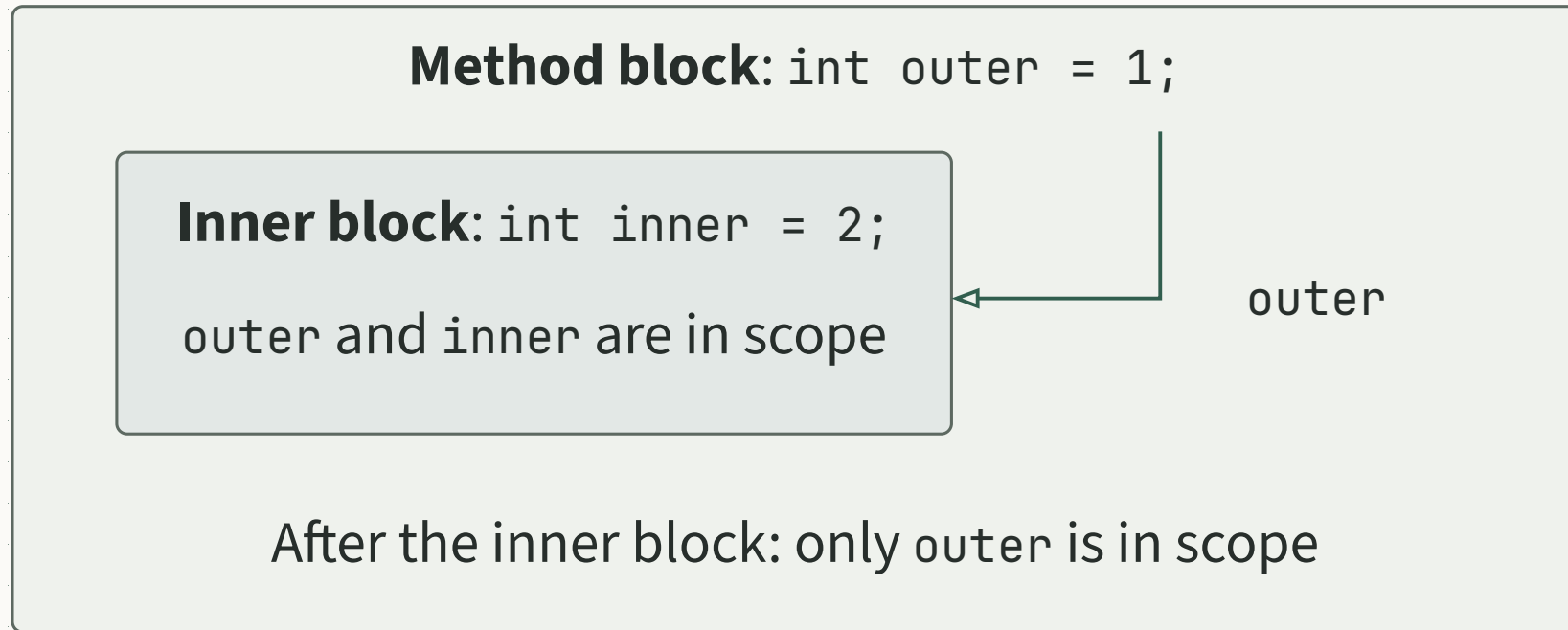
Declaration and Initialization

Declaration and initialization can be combined.

```
1 int a = 5;  
2 float b = 3.5f;  
3 char c = 'A';
```

Java

Scope of Variables



A local variable's scope starts at its declaration and ends with its block.

Type Correctness

Assignments must use compatible types.

```
1 float b = 3.5f;
```

Java

```
2 int a = b; // Compile error: possible loss of precision
```



Memorize

Java does not implicitly narrow a float to an int.

Type Correctness



What differs from C?

Type Correctness

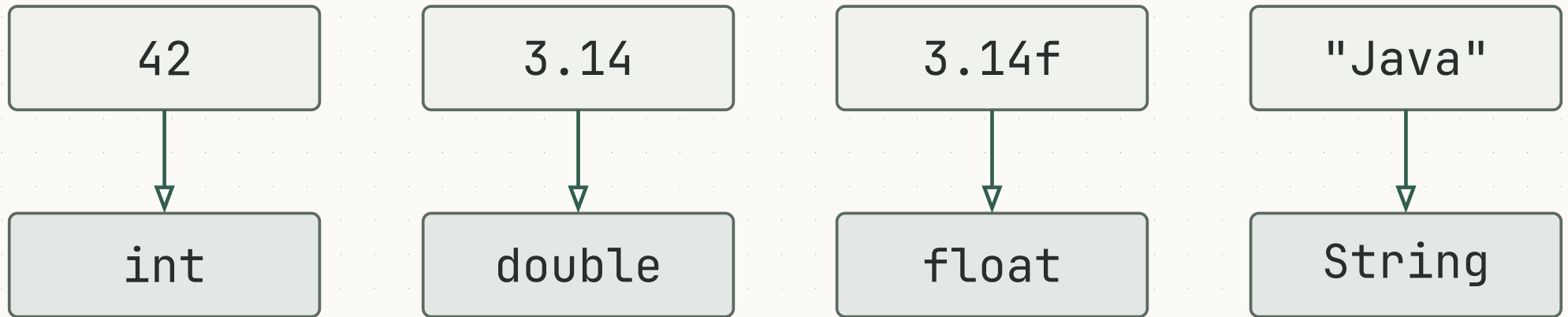


What differs from C?

- Java uses classes and arrays, not C's struct or union.
- Integer widths are fixed; there is no unsigned keyword.
- char is unsigned and stores one 16-bit UTF-16 code unit.
- No pointer arithmetic.

Literals

A **literal** is a value written directly in source code.



Literals



Why does this fail to compile?

```
1 float point = 3.1416;
```

Java

Literals



Why does this fail to compile?

```
1 float point = 3.1416;
```

Java

3.1416 is a double literal. Java does not implicitly narrow it to float.

Literals

Use an f suffix, or choose a double variable.

```
1 float point = 3.1416f;  
2 double precisePoint = 3.1416;
```

Java

Constants

A `final` variable can be assigned only once.

```
1 final double PI = 3.1416;  
2 PI = 3; // Compile error: PI is final
```

Java

Creating Console Output



Task 1

Open an executable class in IntelliJ IDEA. Run this code.

```
1 int age = 24;  
2 System.out.println(24);  
3 System.out.println(age);
```

Java

Creating Console Output

Use + to combine text and values.

```
1 int age = 24;  
2 System.out.println("My age is " + 24);  
3 System.out.println("My age is " + age);
```

Java



Info

In IntelliJ IDEA, type sout and press Tab to insert System.out.println().

Coding Style



What is a coding style?

Coding Style



What is a coding style?

Shared naming and formatting rules make code easier to read and maintain.



Memorize
Coding style is assessed in the exam.

Coding Style: Naming Conventions

Use meaningful English names.

Identifier	Convention	Example
Class	UpperCamelCase	BankAccount
Method / variable	lowerCamelCase	accountBalance
Constant	UPPER_CASE	MAX_SIZE
Package	lowercase	banking

3. Comments and Identifiers

Character Set

Java uses Unicode. Some characters need two UTF-16 code units.



One char = one 16-bit UTF-16 code unit

```
1 System.out.println("\uD83D\uDE00"); //
```



Java

Write course comments in German or English.

Comments



Who will need to read your code?

“Make the code readable? Who else is supposed to read this?”

Comments

- Explain intent, assumptions, and non-obvious decisions.
- Keep comments accurate when code changes.



Memorize

Useful comments explain why, not what the code already says.

Comments



How do line and block comments differ?

Comments



How do line and block comments differ?

```
1 // A line comment ends at the line break.  
2 int distance; // Distance between a and b
```

Java

```
1 /* A block comment can span  
2    several lines. */
```

Java

Identifiers

An **identifier** is a name in your program.

- Start with a Java letter, `_`, or a currency symbol such as `$`.
- Later characters may also be digits.
- Names are case-sensitive; spaces are not allowed.
- Keywords and `true`, `false`, `null` are not names.
- A single `_` is not an identifier (Java 9+).

Identifiers

Reserved keywords

abstract	double	int	super
assert	else	interface	switch
boolean	enum	long	synchronized
break	extends	native	this
byte	final	new	throw
case	finally	package	throws
catch	float	private	transient
char	for	protected	try
class	goto	public	void
const	if	return	volatile
continue	implements	short	while
default	import	static	_
do	instanceof	strictfp	

Contextual keywords such as var have additional restrictions.

Identifiers



Which identifiers are allowed, and why?

```
1 int length;
```

Java

```
2 int länge;
```

```
3 int maxLength;
```

```
4 int max_length;
```

```
5 int _max_length;
```

```
6 int max-length;
```

```
7 int !maxLength;
```

```
1 int 3dlength;
```

Java

```
2 String öpnvKosten;
```

```
3 String €kosten;
```

```
4 String kostenin€;
```

```
5 String €;
```

```
6 int long;
```

```
7 int c.o.s.t;
```

```
8 String @cost;
```

4. Operators

Arithmetic Operators

Operator	Operation	Example
+, -	Unary sign	+a, -a
++, --	Increment / decrement	++count, count--
*, /, %	Multiply / divide / remainder	11 % 4 gives 3
+, -	Add / subtract	a + b, a - b

- Multiplication, division, and remainder bind before addition and subtraction.
- Most binary operators group left to right; parentheses override precedence.
- Integer division discards the fractional part: 7 / 2 is 3.

Increment and Decrement



--a and a-- follow the same order, but subtract 1.

Increment and Decrement



What does this print?

```
1  int a = 1;
2  System.out.println("++a : " + ++a);
3  System.out.println("a   : " + a);
4  System.out.println("a++ : " + a++);
5  System.out.println("--a : " + --a);
6  System.out.println("a-- : " + a--);
```

Java

Comparison Operators

Operator	Meaning
<, ≤	Less than / less than or equal to
>, ≥	Greater than / greater than or equal to
=, ≠	Equal to / not equal to

Comparisons produce a `boolean`. Relational operators bind before equality operators.

Comparison Operators



Do these expressions give the same result?

```
1 int a = 7, b = 4;  
2 boolean parentheses = (a > b) == (a ≤ b);  
3 boolean priorities = a > b == a ≤ b;  
4 System.out.println(parentheses);  
5 System.out.println(priorities);
```

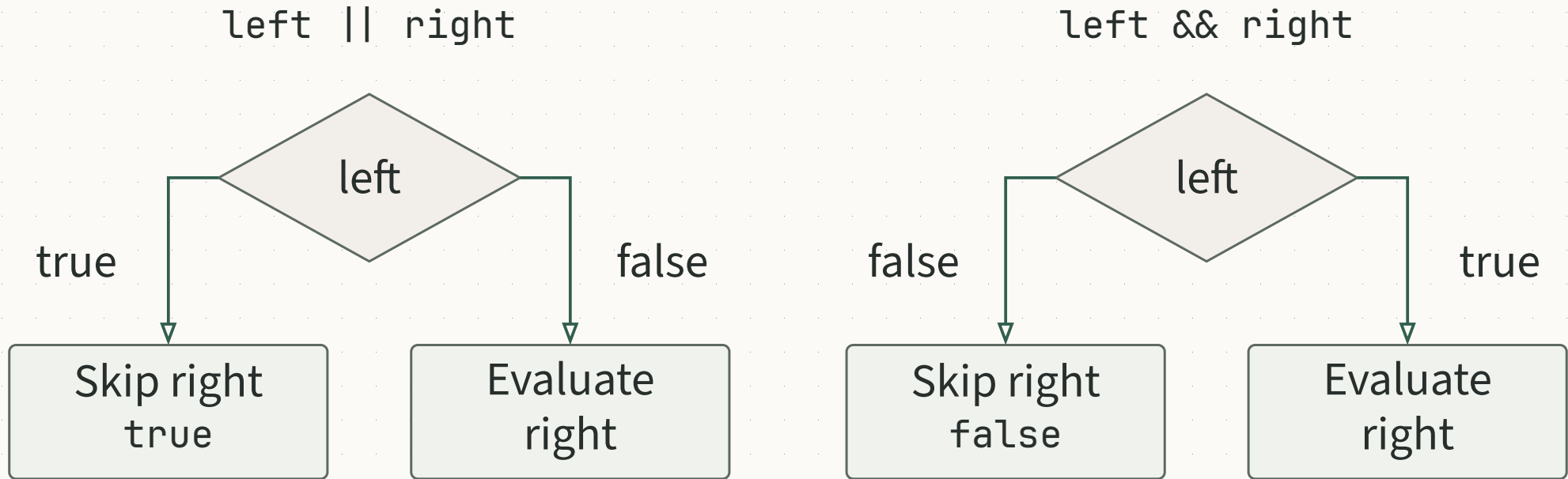
Java

Logical Operators

Operator	Meaning	True when...
!	NOT	the operand is false
^	XOR	the operands differ
&&	AND	both operands are true
	OR	at least one operand is true

For boolean operands, precedence decreases from top to bottom.

Short-Circuit Evaluation



Only `&&` and `||` can skip the right operand; `^` evaluates both.

Short-Circuit Evaluation



What is the final value of a?

```
1 int a = 3, b = 4;  
2 System.out.println((++a == b) || (a++ > b));  
3 System.out.println("a = " + a);
```

Java

Assignment Operators

Assignment	For an int variable a
<code>a = 3</code>	Store 3
<code>a += 2</code>	<code>a = a + 2</code>
<code>a *= 2</code>	<code>a = a * 2</code>
<code>a /= 2</code>	<code>a = a / 2</code>

Also available: `-=`, `%=` and bitwise compound assignments.

Compound assignment evaluates the left operand once and converts the result back to its type.

Assignment Operators



What does this print?

```
1 int a = 1;  
2 a += 2;  
3 System.out.println(a);  
4 System.out.println(a *= --a);  
5 System.out.println(a *= -a++);  
6 System.out.println(a /= 10);
```

Java

5. Type Conversion

Explicit Type Conversion

A narrowing conversion needs an explicit **cast**.

```
1 int a = 80;  
2 byte b = (byte) a;  
3 System.out.println(b);
```

Java



Warning

A cast can lose precision or change the value.

Explicit Type Conversion

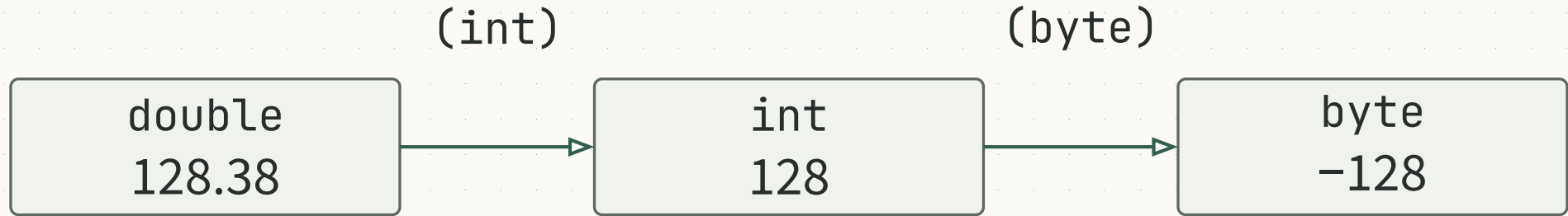


What does each cast produce?

```
1 double a = 128.38;  
2 int b = (int) a;  
3 byte c = (byte) a;  
4 System.out.println("double: " + a);  
5 System.out.println("int : " + b);  
6 System.out.println("byte : " + c);
```

Java

Explicit Type Conversion



Discard fraction

Keep low 8 bits: 10000000



Memorize

A byte ranges from `-128` to `127`. Casting `128` to `byte` produces `-128`.

Implicit Type Conversion

Widening conversions need no cast.

```
1 short a = 71;  
2 int b = (int) a; // Cast is allowed, but redundant  
3 int c = a;
```

Java

Implicit Type Conversion



Which lines compile?

```
1 short a = 1024;  
2 long b = a;  
3 float c = b;
```

Java

Implicit Type Conversion

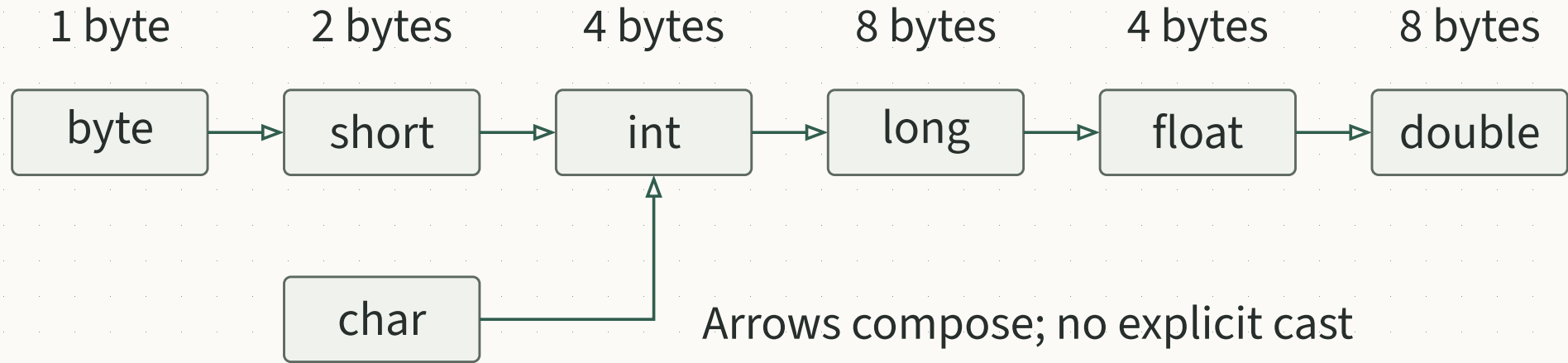


Which lines compile?

```
1 char d = 'A';  
2 short e = d;  
3 int f = d;
```

Java

Implicit Type Conversion



- char: 0...65,535; short: -32,768...32,767. Neither range contains the other.
- int → float and long → float / double can lose precision.

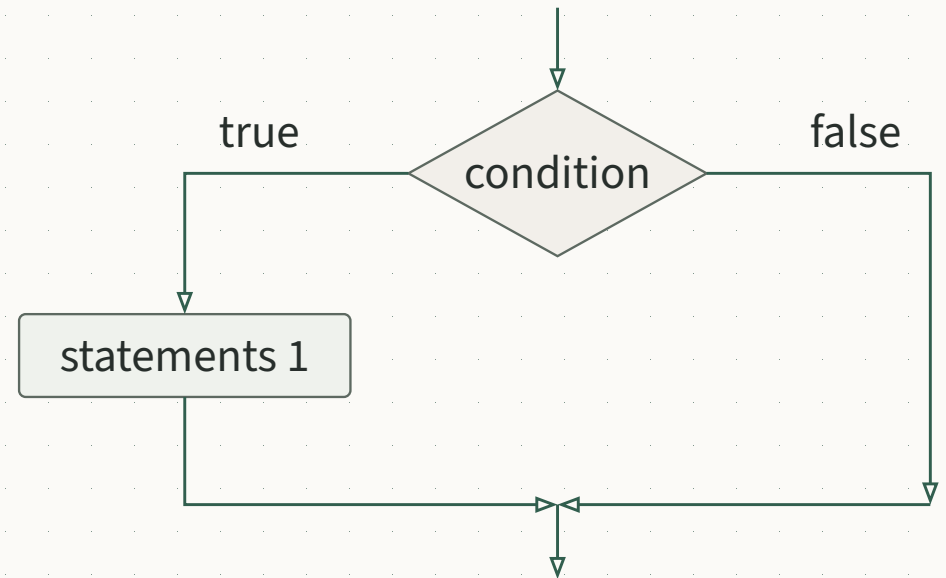
6. Control Structures

if Statement

Execute the body only when the boolean condition is true.

```
1 if (condition) {  
2    // Statements  
3 }
```

Java



if Statement

```
1 int a = 4, b = 8;  
2 int maximum = a;  
3 if (b > maximum) {  
4     maximum = b;  
5 }
```

Java

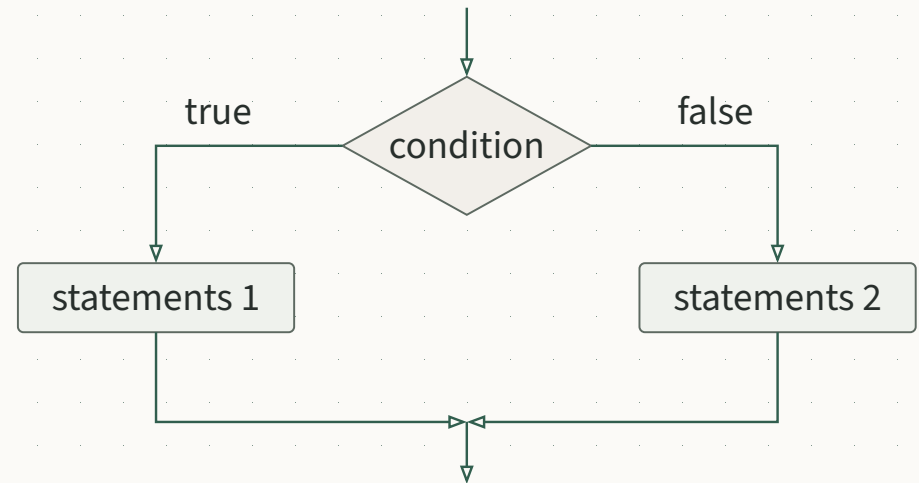
Braces are optional for one statement; keep them for readability.

if-else Statement

Exactly one branch runs.

```
1 if (condition) {  
2    // Statements 1  
3 } else {  
4    // Statements 2  
5 }
```

Java



if-else Statement

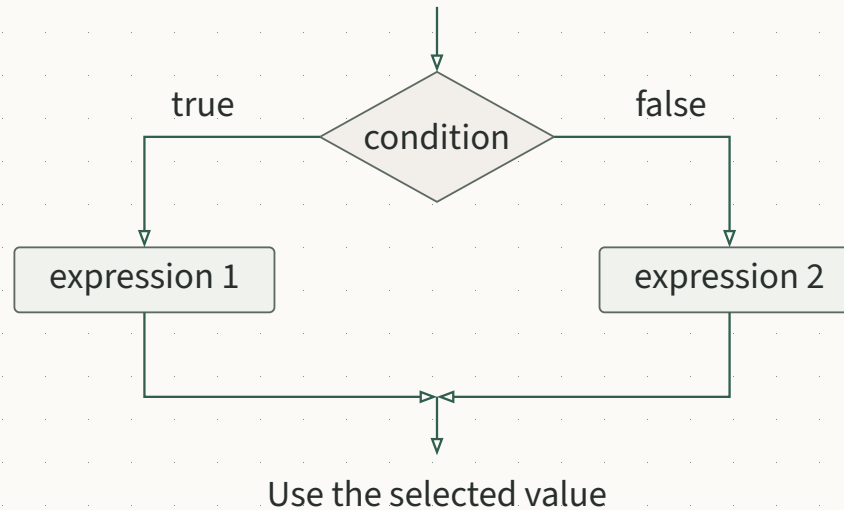
```
1 int a = 4, b = 8;  
2 int maximum;  
3 if (a > b) {  
4     maximum = a;  
5 } else {  
6     maximum = b;  
7 }
```

Java

The ? Operator

Select one of two values.

```
1 int maximum = (a > b) ? a : b;
```

Java

if-else Exercise



Task 2

Given `weekDay` from 1 (Monday) to 7 (Sunday), print the matching activity.



if-else Exercise

```
1 byte weekDay = 3;
2 if (weekDay ≤ 5) {
3     System.out.println("Working");
4 } else if (weekDay = 6) {
5     System.out.println("Shopping");
6 } else if (weekDay = 7) {
7     System.out.println("Resting");
8 }
```

Java

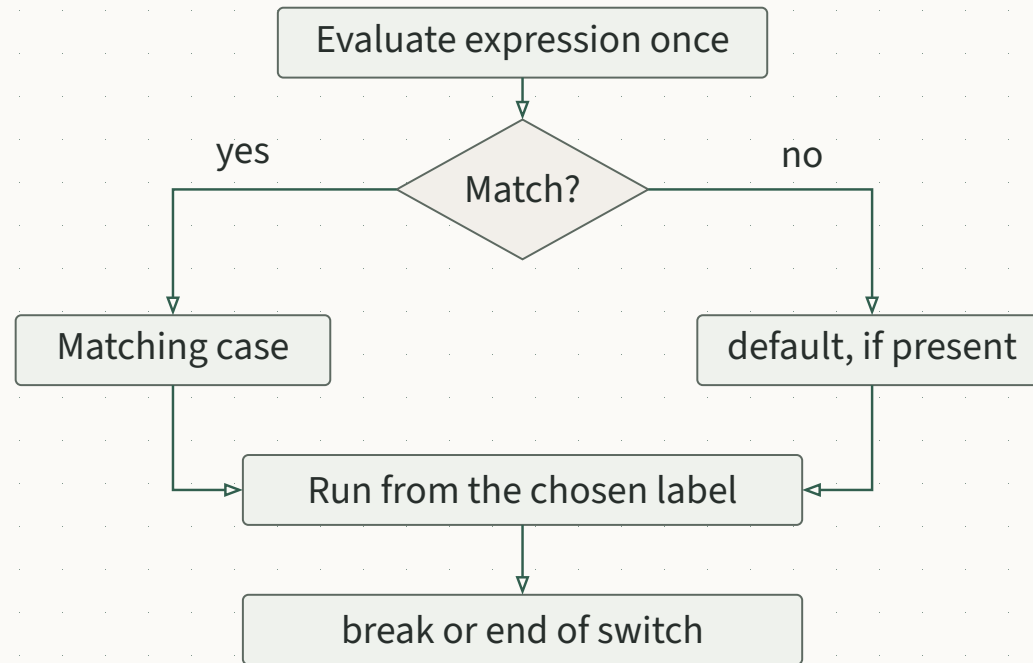
switch Statement

Select an entry point by value.

```
1  switch (expression) {  
2      case 1:  
3          // Statements  
4          break;  
5      default:  
6          // Fallback statements  
7  }
```

Java

switch Statement



No match and no default: skip the switch. Without break, execution falls through subsequent labels.

switch Statement

- Classic switch selectors include byte, short, char, int, their wrappers, enums, and String.
- Several labels can share a body.
- default handles unmatched values; it is optional.



Task 3

Rewrite the weekday exercise using a classic switch statement.

switch Statement

Solution • cases 1–6

```
1  switch (weekDay) {  
2      case 1: case 2: case 3: case 4: case 5:  
3          System.out.println("Working");  
4          break;  
5      case 6:  
6          System.out.println("Shopping");  
7          break;  
}
```

Java

switch Statement

Solution • continued

```
8      case 7:
9          System.out.println("Resting");
10         break;
11     default:
12         System.out.println("Unknown day");
13 }
```

Java

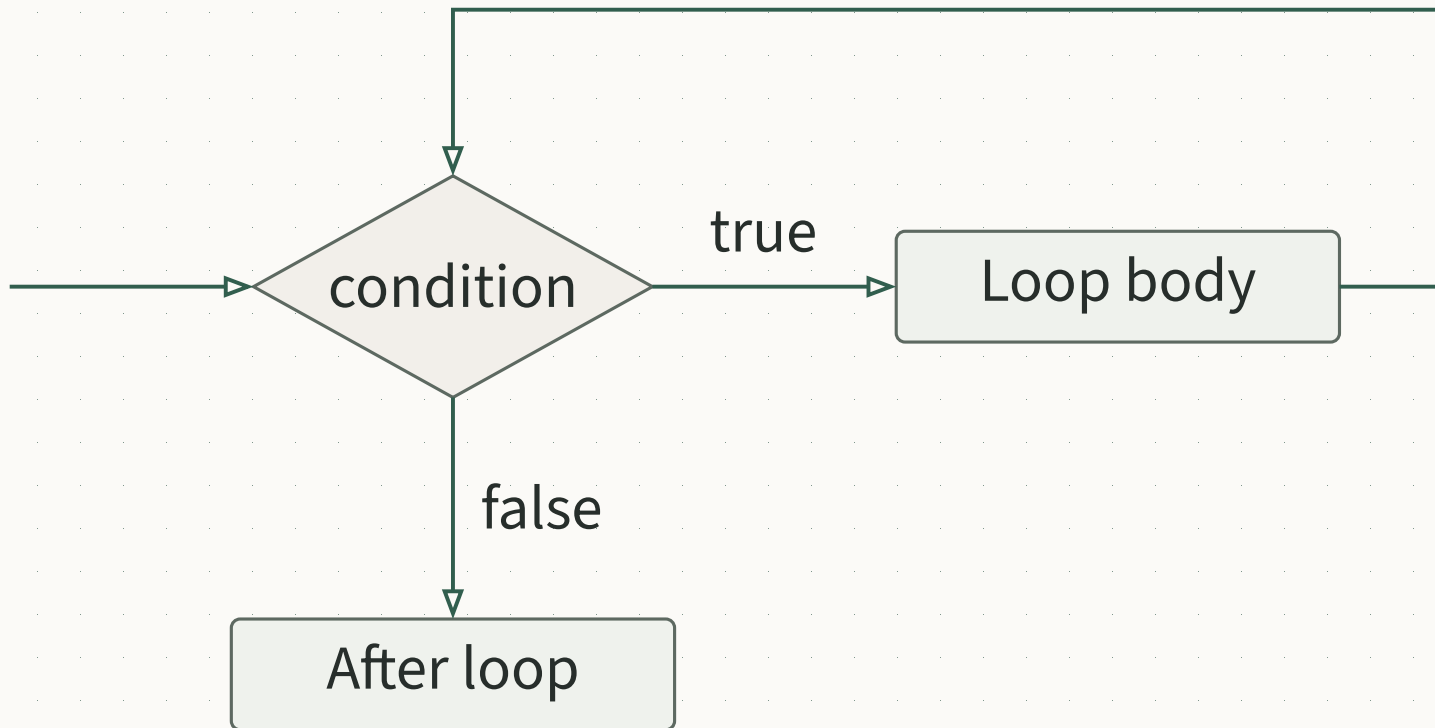
while Loop

Test before each iteration. The body may run **zero times**.

```
1 while (condition) {  
2     // Statements  
3 }
```

Java

while Loop



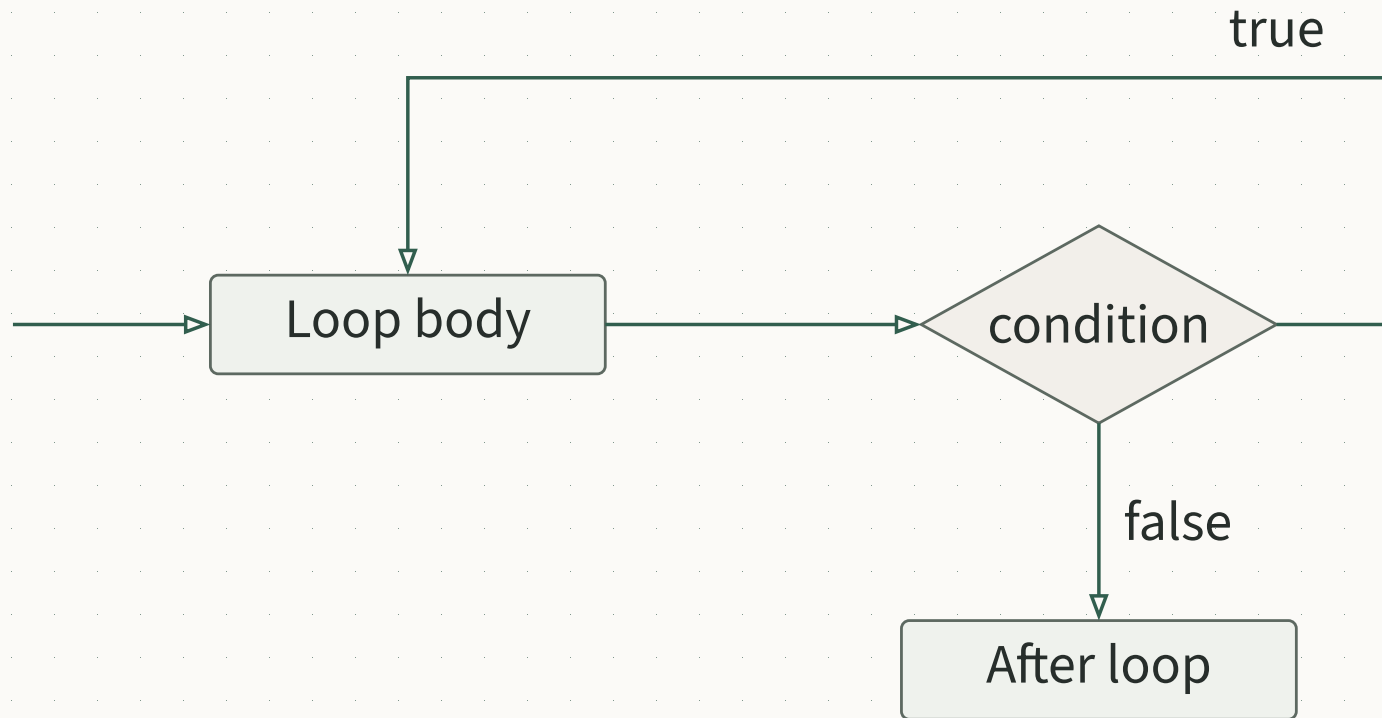
do-while Loop

Test after each iteration. The body runs **at least once**.

```
1 do {  
2     // Statements  
3 } while (condition);
```

Java

do-while Loop



for Loop

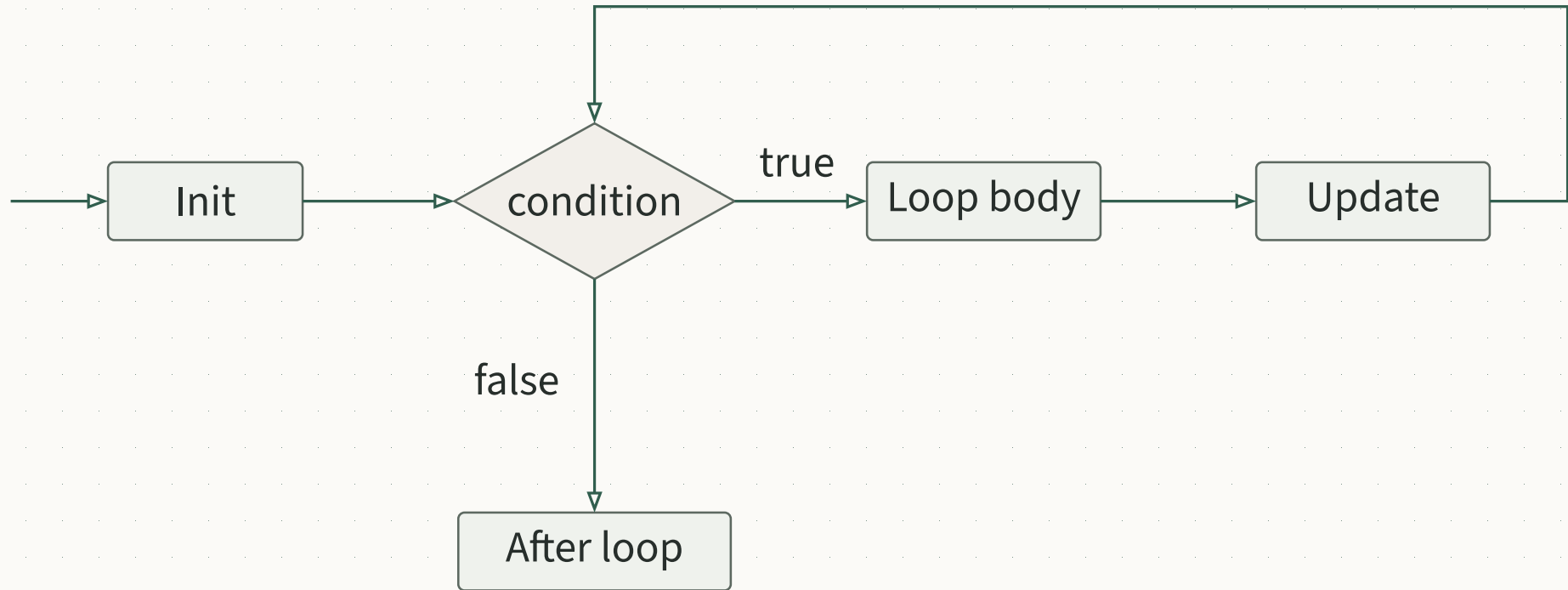
Group initialization, condition, and update in one header.

```
1 for (int i = 0; i < 5; i++) {  
2     System.out.println(i);  
3 }
```

Java

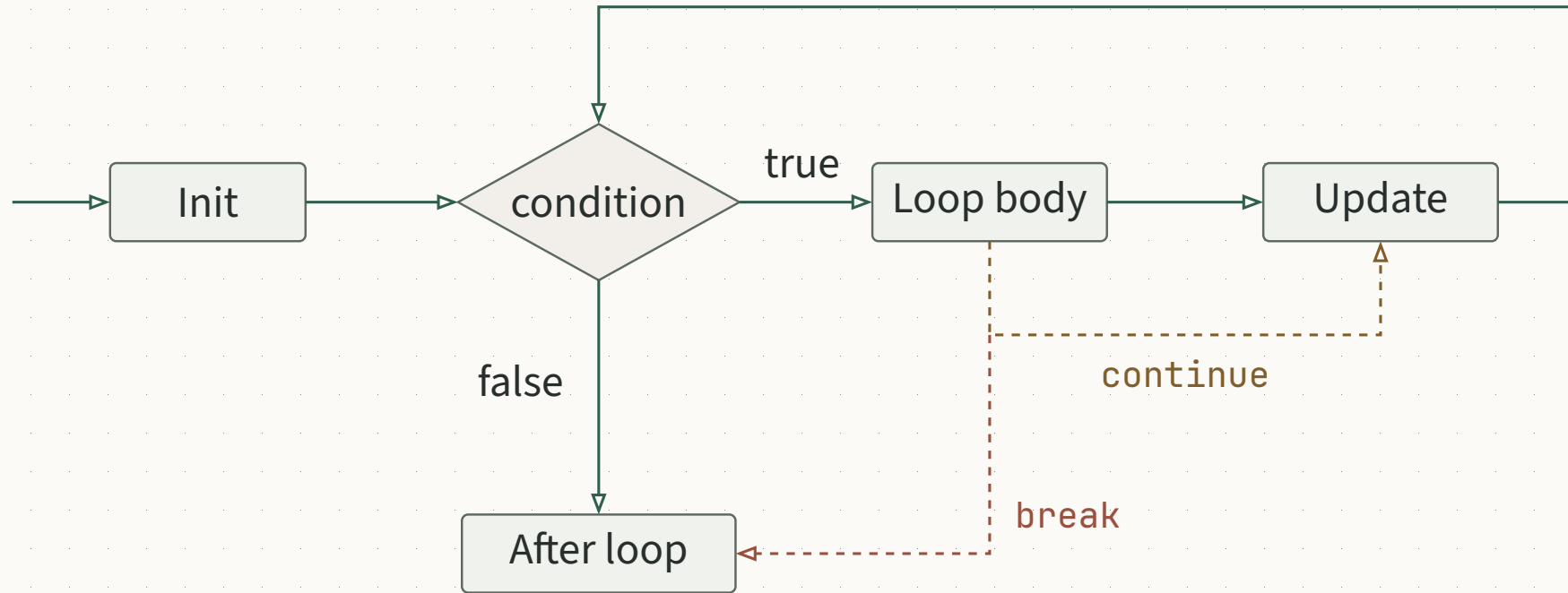
6. Control Structures

for Loop



Init runs once. Update follows each completed iteration.

Jump Statements



break exits the loop. **continue** skips the rest of its body. In a for loop, **continue** runs the update before testing again; in while and do-while, it goes to the condition.

Jump Statements



What does this print?

```
1  for (int i = 0; i ≤ 4; i++) {  
2      if (i = 2) {  
3          break;  
4      }  
5      System.out.println("i = " + i);  
6  }
```

Java

Jump Statements



What changes with continue?

```
1  for (int i = 0; i ≤ 4; i++) {  
2      if (i = 2) {  
3          continue;  
4      }  
5      System.out.println("i = " + i);  
6  }
```

Java

Coding Style

- Open braces on the same line; indent the body consistently.
- Put `else` on the same line as the preceding closing brace.

```
1  if (a > b) {  
2      maximum = a;  
3  } else {  
4      maximum = b;  
5  }
```

Java

7. License Notice

Attribution

- Licensed under CC BY-NC-SA 4.0.
- Based on teaching material by Prof. Dr. Marc Hensel.
- Text, illustrations, and layout have been adapted.
- The original material was provided privately and cannot be linked here.